Source code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.model_selection import train_test_split
from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print(df.head())
print(df.info())

df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)

plt.figure(figsize=(12, 6))
plt.plot(df['Close'], label='Close Price', color='blue')
plt.title('📈 Stock Price (Close) Over Time')
plt.xlabel('Date')
```

```
plt.ylabel('Price')
plt.legend()
plt.grid(True, linestyle='--', linewidth=0.5, alpha=0.7)
plt.show()


scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df[['Close']])
def create_dataset(data, time_step=60):
    X, y = [], []
    if len(data) <= time_step:
        raise ValueError("Not enough data points for the chosen time_step. Reduce time_step or provide more data.")
    for i in range(time_step, len(data)):
        X.append(data[i-time_step:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)


time_step = 2  # Adjust as needed
X, y = create_dataset(scaled_data, time_step)
if X.size == 0 or y.size == 0:
    raise ValueError("Dataset creation resulted in empty X or y. Check your data and time_step.")



X = X.reshape(X.shape[0], X.shape[1], 1)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)


model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=1))


model.compile(optimizer='adam', loss='mean_squared_error')


history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))


test_loss = model.evaluate(X_test, y_test)
print(f"Test Loss: {test_loss}")


predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)


plt.figure(figsize=(14, 7))
plt.plot(df.index[-len(y_test):], scaler.inverse_transform(y_test.reshape(-1, 1)),
        label='Actual Stock Price', color='green', linewidth=2)
plt.plot(df.index[-len(y_test):], predictions,
        label='Predicted Stock Price', color='orange', linewidth=2)
```

```
plt.axvspan(df.index[-len(y_test)], df.index[-1], color='lightgrey', alpha=0.3,
label='Test Period')


plt.title('📈 Stock Price Prediction — Actual vs Predicted', fontsize=16)

plt.xlabel('Date', fontsize=14)

plt.ylabel('Stock Price', fontsize=14)

plt.legend(loc='best', fontsize=12)

plt.grid(True, linestyle='--', linewidth=0.5, alpha=0.7)

plt.tight_layout()

plt.show()


plt.figure(figsize=(12, 6))

plt.plot(history.history['loss'], label='Training Loss', color='blue', linewidth=2)

plt.plot(history.history['val_loss'], label='Validation Loss', color='red', linewidth=2)

plt.title('📉 Model Loss During Training', fontsize=16)

plt.xlabel('Epoch', fontsize=14)

plt.ylabel('Loss (MSE)', fontsize=14)

plt.legend(loc='best', fontsize=12)

plt.grid(True, linestyle='--', linewidth=0.5, alpha=0.7)

plt.tight_layout()

plt.show()


predicted_df = pd.DataFrame({

    'Date': df.index[-len(y_test):],
```

```
    'Predicted Price': predictions.flatten()
})
predicted_df.to_csv('predicted_stock_prices.csv', index=False)
```